Assignment 2 Policy Iteration and Custom Gym Environment

Reinforcement Learning Assignment

Name:

Youssef Sherif Amin-1200968

Abdelrahman Mahmoud Ibrahim-4230179

1. Project Overview

This project implements a custom 5 by 5 Grid Maze environment using Gymnasium and PyGame. The environment contains an agent, one goal cell, and two bad cells. The goal is to reach the goal cell while avoiding the bad cells.

Policy Iteration was used to calculate a policy for a fixed maze layout. The learned policy was then executed in the environment and recorded as an MP4 video.

2. Environment Design

The observation contains eight integers: the row and column of the agent, the goal, and the two bad cells. The agent can take four deterministic actions: right, up, left, and down. An attempt to move outside the grid keeps the agent at the boundary.

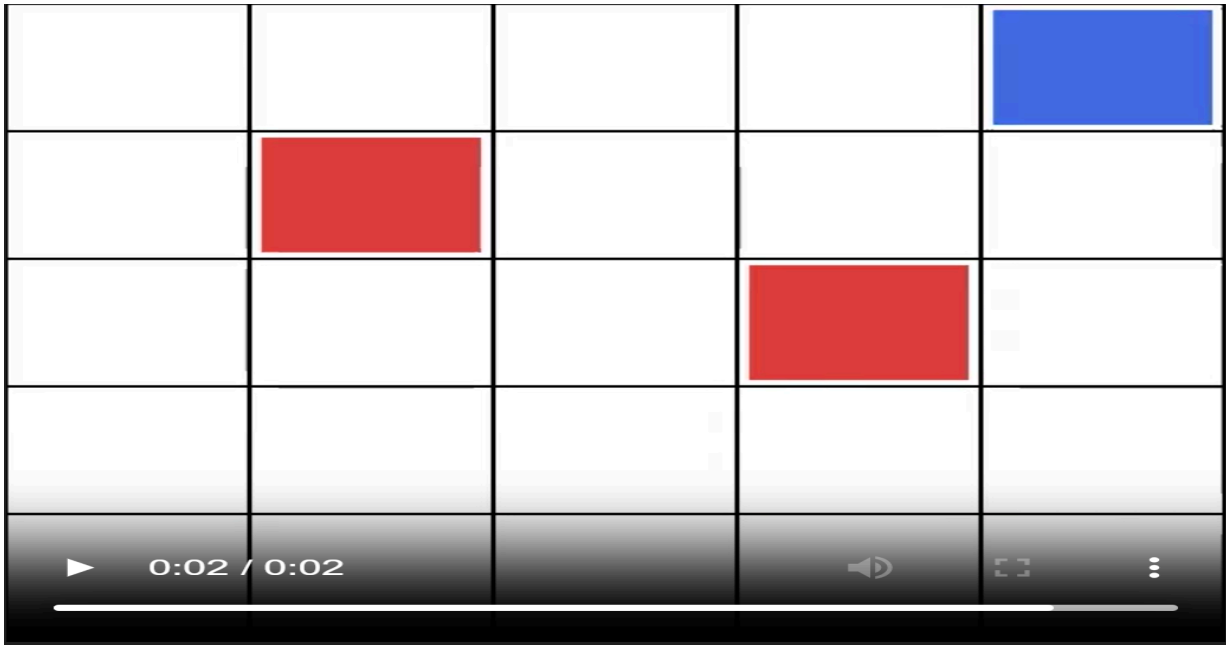
The reward is +1 when the agent reaches the goal, -1 when it reaches a bad cell, and 0 for all other moves. Reaching the goal or a bad cell terminates the episode.

3. Policy Iteration

The Policy Iteration algorithm alternates between policy evaluation and policy improvement. During policy evaluation, it estimates the value of each grid cell under the current policy. During policy improvement, it compares the four possible actions in each non-terminal cell and selects the action with the highest expected value. The process ends when the policy no longer changes.

4. Experimental Result

For the tested maze, the agent started at (4, 0), the goal was at (0, 4), and the two bad cells were at (1, 1) and (2, 3). Policy Iteration converged after 6 iterations. The final policy moved the agent safely to the goal in 8 steps and achieved a final reward of +1.



5. Result Summary

Metric	Result
Maze size	5 by 5 grid
Policy Iteration convergence	6 iterations
Steps to reach goal	8 steps
Final reward	+1.0
Recorded output	policy_iteration-episode-0.mp4

6. Questions and Answers

1. What is the state-space size of the 5x5 Grid Maze problem?

For a fixed 5x5 maze, the agent can occupy 25 grid cells, so the basic state space has 25 states. Each state has four available actions. The full observation also contains the positions of the goal and bad cells, but Policy Iteration in this project plans for one fixed layout at a time.

2. How can Policy Iteration be optimized for the Grid Maze problem?

It can be improved by using vectorized NumPy operations, limiting the number of policy-evaluation sweeps, using sparse transition representations, and starting from a reasonable initial policy. These changes reduce repeated calculations.

3. How many iterations did it take to converge on a stable policy for the 5x5 maze?

For the tested maze layout, Policy Iteration converged after 6 iterations.

4. How does Policy Iteration behave with multiple goal cells?

With multiple goals, Policy Iteration compares the long-term value of reaching each goal. If the goals have the same reward, the agent normally moves toward the closest safe goal because the reward is received sooner and therefore discounted less. For example, if one goal is two moves away and another is far across the maze, the nearby safe goal is usually preferred.

5. Can Policy Iteration work on a 100x100 maze?

It can work, but it becomes more expensive. A 100x100 maze has 10,000 position states, so each policy-evaluation and policy-improvement round must process many more state-action values than in the 5x5 maze.

6. Can Policy Iteration work on a continuous-space maze?

Tabular Policy Iteration cannot directly handle a continuous-space maze because there are infinitely many possible positions. The state space would need to be discretized, or the value function and policy would need to be approximated with another method.

7. Can Policy Iteration work with moving bad cells, such as Pacman ghosts?

Standard Policy Iteration assumes that the environment and transition model are fixed. Moving bad cells make the environment change over time. A stronger model would include the moving objects' positions in the state and repeatedly update or re-plan the policy.

7. Conclusion

The custom Grid Maze environment and Policy Iteration implementation successfully produced a safe policy for the tested layout. The result shows that Dynamic Programming methods can solve small, known environments effectively, while larger, continuous, or changing environments require more scalable approaches.